

NUMPY

```
In [1]: import numpy as np
```

```
In [3]: np.__version__
```

```
Out[3]: '2.3.5'
```

```
In [4]: import sys  
sys.version
```

```
Out[4]: '3.13.9 | packaged by Anaconda, Inc. | (main, Oct 21 2025, 19:09:58) [MSC v.1929 6  
4 bit (AMD64)]'
```

ARANGE ()

- In NumPy, arange means “array range”

It is used to create an array of numbers within a given range, similar to Python’s range(), but it returns a NumPy array instead of a range object.

- in Arange() 1st arg should be less than 2nd arg

```
In [5]: np.arange(10)
```

```
Out[5]: array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
In [6]: np.arange (20)
```

```
Out[6]: array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16,  
17, 18, 19])
```

```
In [7]: np.arange (30)
```

```
Out[7]: array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16,  
17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29])
```

```
In [8]: np.arange (50)
```

```
Out[8]: array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16,  
17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33,  
34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49])
```

```
In [9]: np.arange (10,20,)
```

```
Out[9]: array([10, 11, 12, 13, 14, 15, 16, 17, 18, 19])
```

```
In [10]: np.arange (10,20,5)
```

```
Out[10]: array([10, 15])
```

Creating Arrays

```
In [11]: my_list = [0,1,2,3,4,5]
         my_list
```

```
Out[11]: [0, 1, 2, 3, 4, 5]
```

```
In [12]: type(my_list)
```

```
Out[12]: list
```

```
In [13]: arr = np.array(my_list)
         arr
```

```
Out[13]: array([0, 1, 2, 3, 4, 5])
```

```
In [14]: type(arr)
```

```
Out[14]: numpy.ndarray
```

```
In [15]: print(my_list)
         print(arr)
```

```
[0, 1, 2, 3, 4, 5]
[0 1 2 3 4 5]
```

```
In [16]: print(type(my_list))
         print(type(arr))
```

```
<class 'list'>
<class 'numpy.ndarray'>
```

```
In [17]: np.arange(10)
```

```
Out[17]: array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
In [18]: np.arange(10,20)
```

```
Out[18]: array([10, 11, 12, 13, 14, 15, 16, 17, 18, 19])
```

```
In [19]: np.arange(20,10)
```

```
Out[19]: array([], dtype=int64)
```

```
In [20]: np.arange(10,50,5)
```

```
Out[20]: array([10, 15, 20, 25, 30, 35, 40, 45])
```

np.zeros()

- Creates an array filled with 0s.

```
In [21]: np.zeros(5)
```

```
Out[21]: array([0., 0., 0., 0., 0.])
```

```
In [22]: np.zeros(5, dtype = int)
```

```
Out[22]: array([0, 0, 0, 0, 0])
```

```
In [23]: np.zeros(10)
```

```
Out[23]: array([0., 0., 0., 0., 0., 0., 0., 0., 0., 0.])
```

```
In [24]: np.zeros(10, dtype = int)
```

```
Out[24]: array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0])
```

```
In [25]: np.zeros((2,2))
```

```
Out[25]: array([[0., 0.],  
               [0., 0.]])
```

```
In [26]: np.zeros((2,3))
```

```
Out[26]: array([[0., 0., 0.],  
               [0., 0., 0.]])
```

```
In [27]: np.zeros((10,5), dtype = int)
```

```
Out[27]: array([[0, 0, 0, 0, 0],  
               [0, 0, 0, 0, 0],  
               [0, 0, 0, 0, 0],  
               [0, 0, 0, 0, 0],  
               [0, 0, 0, 0, 0],  
               [0, 0, 0, 0, 0],  
               [0, 0, 0, 0, 0],  
               [0, 0, 0, 0, 0],  
               [0, 0, 0, 0, 0],  
               [0, 0, 0, 0, 0]])
```

np.ones()

- Creates an array filled with 1s.

```
In [28]: np.ones(4)
```

```
Out[28]: array([1., 1., 1., 1.])
```

```
In [29]: np.ones((4), dtype = int)
```

```
Out[29]: array([1, 1, 1, 1])
```

```
In [30]: np.ones((4,5))
```

```
Out[30]: array([[1., 1., 1., 1., 1.],
               [1., 1., 1., 1., 1.],
               [1., 1., 1., 1., 1.],
               [1., 1., 1., 1., 1.]])
```

```
In [31]: np.ones((10,5),dtype = int)
```

```
Out[31]: array([[1, 1, 1, 1, 1],
               [1, 1, 1, 1, 1],
               [1, 1, 1, 1, 1],
               [1, 1, 1, 1, 1],
               [1, 1, 1, 1, 1],
               [1, 1, 1, 1, 1],
               [1, 1, 1, 1, 1],
               [1, 1, 1, 1, 1],
               [1, 1, 1, 1, 1],
               [1, 1, 1, 1, 1]])
```

```
In [32]: random(2)
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[32], line 1
----> 1 random(2)

NameError: name 'random' is not defined
```

```
In [33]: random.rand(2)
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[33], line 1
----> 1 random.rand(2)

NameError: name 'random' is not defined
```

```
In [34]: np.random(3)
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[34], line 1
----> 1 np.random(3)

TypeError: 'module' object is not callable
```

np.random.rand()

- Generates random floating-point numbers between 0 and 1

```
In [35]: np.random.rand(4)
```

```
Out[35]: array([0.67400442, 0.31920411, 0.15275349, 0.39225799])
```



```
In [36]: np.random.rand(5)
```

```
Out[36]: array([0.09203389, 0.05229463, 0.40194264, 0.84371889, 0.85468586])
```

```
In [37]: np.random.rand(10)
```

```
Out[37]: array([0.52466528, 0.56013737, 0.70071689, 0.79707762, 0.31198716,  
                0.38584205, 0.5333924 , 0.28171246, 0.01987435, 0.2979175 ])
```

2D Example

```
In [38]: np.random.rand(3,2,)
```

```
Out[38]: array([[0.84417005, 0.44829514],  
                [0.1609653 , 0.57706445],  
                [0.76614869, 0.21490987]])
```

np.random.randint()

- Generates random integers within a given range.

```
In [39]: np.random.randint(5)
```

```
Out[39]: 1
```

```
In [40]: np.random.randint(5,10)
```

```
Out[40]: 8
```

```
In [41]: np.random.randint(5,10,3).astype(int) # Means it gives 3 random numbers from (5 - 10)
```

```
Out[41]: array([6, 5, 5])
```

```
In [42]: np.random.randint(5,10,5) #It gives 5 random integers from (5-10)
```

```
Out[42]: array([9, 9, 9, 6, 7], dtype=int32)
```

```
In [43]: np.random.randint (10,15,9)
```

```
Out[43]: array([10, 13, 10, 14, 14, 14, 13, 14, 11], dtype=int32)
```

```
In [44]: np.random.randint(10,5,5) ## Low >= high ( 1st arg should be less than 2 nd arg)
```

```

-----
ValueError                                Traceback (most recent call last)
Cell In[44], line 1
----> 1 np.random.randint(10,5,5)    ## low >= high ( 1st arg should be less than 2
nd arg)

File numpy/random/mtrand.pyx:794, in numpy.random.mtrand.RandomState.randint()

File numpy/random/_bounded_integers.pyx:1425, in numpy.random._bounded_integers._rand_int32()

ValueError: low >= high

```

```
In [45]: np.random.randint(5,10,[5,7])    # It gives random 5 rows and 7 columns from the 5-1
```

```
Out[45]: array([[9, 6, 6, 6, 6, 7, 6],
               [8, 8, 8, 6, 8, 5, 5],
               [5, 8, 7, 7, 9, 6, 7],
               [5, 9, 9, 7, 7, 8, 8],
               [8, 5, 9, 9, 8, 7, 7]], dtype=int32)
```

```
In [46]: np.random.randint(10,15,(3,2)).astype(int)
```

```
Out[46]: array([[12, 14],
               [13, 13],
               [13, 10]])
```

```
In [47]: np.random.randint(25,50,(5, 10)).astype(int)
```

```
Out[47]: array([[45, 30, 37, 47, 26, 47, 28, 40, 25, 25],
               [47, 26, 48, 48, 49, 44, 36, 45, 45, 25],
               [35, 44, 44, 33, 39, 37, 34, 48, 45, 35],
               [31, 34, 38, 28, 40, 25, 48, 25, 33, 34],
               [43, 41, 27, 41, 35, 30, 42, 44, 34, 41]])
```

```
In [48]: np.random.randint(10,2)          ## 1st arg is should be less than 2nd arg
```

```

-----
ValueError                                Traceback (most recent call last)
Cell In[48], line 1
----> 1 np.random.randint(10,2)          ## 1st arg is should be less than 2nd
arg

File numpy/random/mtrand.pyx:794, in numpy.random.mtrand.RandomState.randint()

File numpy/random/_bounded_integers.pyx:1425, in numpy.random._bounded_integers._rand_int32()

ValueError: low >= high

```

```
In [49]: np.random.randint(-10,2)
```

```
Out[49]: -6
```

```
In [50]: np.random.randint(-10,2,5)
```

```
Out[50]: array([-1, -6, -7, -9, -6], dtype=int32)
```

```
In [51]: np.random.randint(10,40,(10,10))
```

```
Out[51]: array([[29, 18, 23, 10, 39, 27, 11, 28, 26, 25],
 [18, 30, 13, 11, 19, 20, 18, 34, 18, 19],
 [38, 26, 33, 26, 23, 30, 32, 30, 26, 33],
 [30, 39, 12, 26, 36, 31, 37, 18, 33, 14],
 [28, 28, 33, 14, 11, 20, 37, 30, 24, 35],
 [18, 26, 13, 35, 32, 10, 11, 18, 20, 17],
 [29, 24, 30, 26, 27, 27, 15, 24, 11, 29],
 [18, 30, 21, 32, 26, 13, 30, 13, 17, 23],
 [10, 20, 25, 26, 18, 31, 28, 32, 38, 23],
 [24, 37, 19, 26, 22, 11, 14, 21, 14, 21]], dtype=int32)
```

```
In [52]: arr1 = np.random.randint(10,40,(10,10)).astype(int)
arr1
```

```
Out[52]: array([[36, 12, 39, 10, 25, 18, 30, 25, 36, 34],
 [17, 11, 14, 28, 22, 38, 17, 29, 31, 12],
 [31, 35, 14, 27, 25, 24, 37, 25, 15, 38],
 [30, 37, 32, 37, 39, 24, 20, 30, 24, 11],
 [33, 17, 24, 25, 31, 17, 28, 36, 21, 21],
 [37, 38, 28, 23, 15, 32, 28, 17, 28, 11],
 [17, 37, 37, 17, 36, 17, 27, 26, 32, 21],
 [19, 10, 30, 15, 14, 25, 31, 10, 26, 35],
 [26, 31, 14, 16, 22, 22, 35, 37, 20, 27],
 [30, 32, 17, 15, 20, 25, 25, 21, 29, 29]])
```

Indexing and slicing in array

```
In [53]: arr1
```

```
Out[53]: array([[36, 12, 39, 10, 25, 18, 30, 25, 36, 34],
 [17, 11, 14, 28, 22, 38, 17, 29, 31, 12],
 [31, 35, 14, 27, 25, 24, 37, 25, 15, 38],
 [30, 37, 32, 37, 39, 24, 20, 30, 24, 11],
 [33, 17, 24, 25, 31, 17, 28, 36, 21, 21],
 [37, 38, 28, 23, 15, 32, 28, 17, 28, 11],
 [17, 37, 37, 17, 36, 17, 27, 26, 32, 21],
 [19, 10, 30, 15, 14, 25, 31, 10, 26, 35],
 [26, 31, 14, 16, 22, 22, 35, 37, 20, 27],
 [30, 32, 17, 15, 20, 25, 25, 21, 29, 29]])
```

```
In [54]: arr1[:]
```

```
Out[54]: array([[36, 12, 39, 10, 25, 18, 30, 25, 36, 34],
               [17, 11, 14, 28, 22, 38, 17, 29, 31, 12],
               [31, 35, 14, 27, 25, 24, 37, 25, 15, 38],
               [30, 37, 32, 37, 39, 24, 20, 30, 24, 11],
               [33, 17, 24, 25, 31, 17, 28, 36, 21, 21],
               [37, 38, 28, 23, 15, 32, 28, 17, 28, 11],
               [17, 37, 37, 17, 36, 17, 27, 26, 32, 21],
               [19, 10, 30, 15, 14, 25, 31, 10, 26, 35],
               [26, 31, 14, 16, 22, 22, 35, 37, 20, 27],
               [30, 32, 17, 15, 20, 25, 25, 21, 29, 29]])
```

```
In [55]: len(arr1)
```

```
Out[55]: 10
```

```
In [56]: arr1.shape      ## Gives dimension of an array row and column count
```

```
Out[56]: (10, 10)
```

```
In [57]: arr1[-4]
```

```
Out[57]: array([17, 37, 37, 17, 36, 17, 27, 26, 32, 21])
```

```
In [58]: arr1[-6]
```

```
Out[58]: array([33, 17, 24, 25, 31, 17, 28, 36, 21, 21])
```

```
In [59]: arr1[-3]
```

```
Out[59]: array([19, 10, 30, 15, 14, 25, 31, 10, 26, 35])
```

```
In [60]: arr1[:, -4]      ### (-4 → 4th row from the bottom ); (: → all columns)
```

```
Out[60]: array([30, 17, 37, 20, 28, 28, 27, 31, 35, 25])
```

```
In [61]: arr1[-4, :]      ### (: → all rows); (-4 → 4th column from the right)
```

```
Out[61]: array([17, 37, 37, 17, 36, 17, 27, 26, 32, 21])
```

```
In [62]: arr1
```

```
Out[62]: array([[36, 12, 39, 10, 25, 18, 30, 25, 36, 34],
               [17, 11, 14, 28, 22, 38, 17, 29, 31, 12],
               [31, 35, 14, 27, 25, 24, 37, 25, 15, 38],
               [30, 37, 32, 37, 39, 24, 20, 30, 24, 11],
               [33, 17, 24, 25, 31, 17, 28, 36, 21, 21],
               [37, 38, 28, 23, 15, 32, 28, 17, 28, 11],
               [17, 37, 37, 17, 36, 17, 27, 26, 32, 21],
               [19, 10, 30, 15, 14, 25, 31, 10, 26, 35],
               [26, 31, 14, 16, 22, 22, 35, 37, 20, 27],
               [30, 32, 17, 15, 20, 25, 25, 21, 29, 29]])
```

```
In [63]: arr1[3]
```

```
Out[63]: array([30, 37, 32, 37, 39, 24, 20, 30, 24, 11])
```

```
In [64]: arr1 [2:7]
```

```
Out[64]: array([[31, 35, 14, 27, 25, 24, 37, 25, 15, 38],
                [30, 37, 32, 37, 39, 24, 20, 30, 24, 11],
                [33, 17, 24, 25, 31, 17, 28, 36, 21, 21],
                [37, 38, 28, 23, 15, 32, 28, 17, 28, 11],
                [17, 37, 37, 17, 36, 17, 27, 26, 32, 21]])
```

```
In [65]: arr1[2, 7]
```

```
Out[65]: np.int64(25)
```

```
In [66]: arr2 = np.random.randint(1,10,(3,3)) .astype(int)
arr2
```

```
Out[66]: array([[5, 1, 6],
                [9, 1, 7],
                [4, 6, 7]])
```

```
In [67]: arr2[2]
```

```
Out[67]: array([4, 6, 7])
```

```
In [68]: arr2[1:2]
```

```
Out[68]: array([[9, 1, 7]])
```

```
In [69]: arr2[1,2]
```

```
Out[69]: np.int64(7)
```

```
In [70]: arr2 [::-1]
```

```
Out[70]: array([[4, 6, 7],
                [9, 1, 7],
                [5, 1, 6]])
```

```
In [71]: arr1
```

```
Out[71]: array([[36, 12, 39, 10, 25, 18, 30, 25, 36, 34],
                [17, 11, 14, 28, 22, 38, 17, 29, 31, 12],
                [31, 35, 14, 27, 25, 24, 37, 25, 15, 38],
                [30, 37, 32, 37, 39, 24, 20, 30, 24, 11],
                [33, 17, 24, 25, 31, 17, 28, 36, 21, 21],
                [37, 38, 28, 23, 15, 32, 28, 17, 28, 11],
                [17, 37, 37, 17, 36, 17, 27, 26, 32, 21],
                [19, 10, 30, 15, 14, 25, 31, 10, 26, 35],
                [26, 31, 14, 16, 22, 22, 35, 37, 20, 27],
                [30, 32, 17, 15, 20, 25, 25, 21, 29, 29]])
```

```
In [72]: arr1[:5]
```

```
Out[72]: array([[36, 12, 39, 10, 25, 18, 30, 25, 36, 34],
               [17, 11, 14, 28, 22, 38, 17, 29, 31, 12],
               [31, 35, 14, 27, 25, 24, 37, 25, 15, 38],
               [30, 37, 32, 37, 39, 24, 20, 30, 24, 11],
               [33, 17, 24, 25, 31, 17, 28, 36, 21, 21]])
```

```
In [73]: arr1[-5,3]
```

```
Out[73]: np.int64(23)
```

```
In [74]: arr1[-5,-3]
```

```
Out[74]: np.int64(17)
```

```
In [75]: arr1[0:-2]
```

```
Out[75]: array([[36, 12, 39, 10, 25, 18, 30, 25, 36, 34],
               [17, 11, 14, 28, 22, 38, 17, 29, 31, 12],
               [31, 35, 14, 27, 25, 24, 37, 25, 15, 38],
               [30, 37, 32, 37, 39, 24, 20, 30, 24, 11],
               [33, 17, 24, 25, 31, 17, 28, 36, 21, 21],
               [37, 38, 28, 23, 15, 32, 28, 17, 28, 11],
               [17, 37, 37, 17, 36, 17, 27, 26, 32, 21],
               [19, 10, 30, 15, 14, 25, 31, 10, 26, 35]])
```

```
In [76]: arr1[0,2]
```

```
Out[76]: np.int64(39)
```

reshape

- Reshaping changes the dimensions of a NumPy array while keeping the total number of elements the same.

```
In [77]: arr
```

```
Out[77]: array([0, 1, 2, 3, 4, 5])
```

```
In [78]: arr.reshape(2,3)
```

```
Out[78]: array([[0, 1, 2],
               [3, 4, 5]])
```

```
In [79]: arr.reshape(3,3)    # (can't make in 6 shape where it is 3* 3)
```

```
-----
ValueError                                Traceback (most recent call last)
Cell In[79], line 1
----> 1 arr.reshape(3,3)    # (can't make in 6 shape where it is 3* 3)

ValueError: cannot reshape array of size 6 into shape (3,3)
```

```
In [80]: arr.reshape(3,2)
```

```
Out[80]: array([[0, 1],
               [2, 3],
               [4, 5]])
```

```
In [81]: arr.reshape(1,6)
```

```
Out[81]: array([[0, 1, 2, 3, 4, 5]])
```

```
In [82]: arr.reshape(6,1)
```

```
Out[82]: array([[0],
               [1],
               [2],
               [3],
               [4],
               [5]])
```

NumPy Array creation and Reshaping

```
In [83]: mat = np.arange(0,100).reshape (10,10)
mat
```

```
Out[83]: array([[ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9],
               [10, 11, 12, 13, 14, 15, 16, 17, 18, 19],
               [20, 21, 22, 23, 24, 25, 26, 27, 28, 29],
               [30, 31, 32, 33, 34, 35, 36, 37, 38, 39],
               [40, 41, 42, 43, 44, 45, 46, 47, 48, 49],
               [50, 51, 52, 53, 54, 55, 56, 57, 58, 59],
               [60, 61, 62, 63, 64, 65, 66, 67, 68, 69],
               [70, 71, 72, 73, 74, 75, 76, 77, 78, 79],
               [80, 81, 82, 83, 84, 85, 86, 87, 88, 89],
               [90, 91, 92, 93, 94, 95, 96, 97, 98, 99]])
```

```
In [84]: row = 4
col = 5
```

```
In [85]: mat[row,col]
```

```
Out[85]: np.int64(45)
```

```
In [86]: mat[6]
```

```
Out[86]: array([60, 61, 62, 63, 64, 65, 66, 67, 68, 69])
```

```
In [87]: mat[:,6]
```

```
Out[87]: array([ 6, 16, 26, 36, 46, 56, 66, 76, 86, 96])
```

```
In [88]: mat[:, -3]
```

```
Out[88]: array([ 7, 17, 27, 37, 47, 57, 67, 77, 87, 97])
```

```
In [89]: mat
```

```
Out[89]: array([[ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9],
               [10, 11, 12, 13, 14, 15, 16, 17, 18, 19],
               [20, 21, 22, 23, 24, 25, 26, 27, 28, 29],
               [30, 31, 32, 33, 34, 35, 36, 37, 38, 39],
               [40, 41, 42, 43, 44, 45, 46, 47, 48, 49],
               [50, 51, 52, 53, 54, 55, 56, 57, 58, 59],
               [60, 61, 62, 63, 64, 65, 66, 67, 68, 69],
               [70, 71, 72, 73, 74, 75, 76, 77, 78, 79],
               [80, 81, 82, 83, 84, 85, 86, 87, 88, 89],
               [90, 91, 92, 93, 94, 95, 96, 97, 98, 99]])
```

```
In [90]: mat[: -3]
```

```
Out[90]: array([[ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9],
               [10, 11, 12, 13, 14, 15, 16, 17, 18, 19],
               [20, 21, 22, 23, 24, 25, 26, 27, 28, 29],
               [30, 31, 32, 33, 34, 35, 36, 37, 38, 39],
               [40, 41, 42, 43, 44, 45, 46, 47, 48, 49],
               [50, 51, 52, 53, 54, 55, 56, 57, 58, 59],
               [60, 61, 62, 63, 64, 65, 66, 67, 68, 69]])
```

Step slicing in array

```
In [91]: mat
```

```
Out[91]: array([[ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9],
               [10, 11, 12, 13, 14, 15, 16, 17, 18, 19],
               [20, 21, 22, 23, 24, 25, 26, 27, 28, 29],
               [30, 31, 32, 33, 34, 35, 36, 37, 38, 39],
               [40, 41, 42, 43, 44, 45, 46, 47, 48, 49],
               [50, 51, 52, 53, 54, 55, 56, 57, 58, 59],
               [60, 61, 62, 63, 64, 65, 66, 67, 68, 69],
               [70, 71, 72, 73, 74, 75, 76, 77, 78, 79],
               [80, 81, 82, 83, 84, 85, 86, 87, 88, 89],
               [90, 91, 92, 93, 94, 95, 96, 97, 98, 99]])
```

```
In [92]: mat[0:1:3]
```

```
Out[92]: array([[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]])
```

```
In [93]: mat[0:2:5]
```

```
Out[93]: array([[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]])
```

```
In [94]: mat[3:6:2]
```

```
Out[94]: array([[30, 31, 32, 33, 34, 35, 36, 37, 38, 39],
               [50, 51, 52, 53, 54, 55, 56, 57, 58, 59]])
```

How to print a specific values from array

```
In [95]: mat
```



```
Out[95]: array([[ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9],
               [10, 11, 12, 13, 14, 15, 16, 17, 18, 19],
               [20, 21, 22, 23, 24, 25, 26, 27, 28, 29],
               [30, 31, 32, 33, 34, 35, 36, 37, 38, 39],
               [40, 41, 42, 43, 44, 45, 46, 47, 48, 49],
               [50, 51, 52, 53, 54, 55, 56, 57, 58, 59],
               [60, 61, 62, 63, 64, 65, 66, 67, 68, 69],
               [70, 71, 72, 73, 74, 75, 76, 77, 78, 79],
               [80, 81, 82, 83, 84, 85, 86, 87, 88, 89],
               [90, 91, 92, 93, 94, 95, 96, 97, 98, 99]])
```

```
In [96]: mat[1:2, 2:4]
```

```
Out[96]: array([[12, 13]])
```

```
In [97]: mat[2:4, 1:4]
```

```
Out[97]: array([[21, 22, 23],
               [31, 32, 33]])
```

```
In [98]: mat[3:6, 3:6]
```

```
Out[98]: array([[33, 34, 35],
               [43, 44, 45],
               [53, 54, 55]])
```

Numpy Masking ---(Filters)

- A condition is applied to an array → It creates a boolean mask → NumPy uses that mask to filter elements.

```
In [99]: mat
```

```
Out[99]: array([[ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9],
               [10, 11, 12, 13, 14, 15, 16, 17, 18, 19],
               [20, 21, 22, 23, 24, 25, 26, 27, 28, 29],
               [30, 31, 32, 33, 34, 35, 36, 37, 38, 39],
               [40, 41, 42, 43, 44, 45, 46, 47, 48, 49],
               [50, 51, 52, 53, 54, 55, 56, 57, 58, 59],
               [60, 61, 62, 63, 64, 65, 66, 67, 68, 69],
               [70, 71, 72, 73, 74, 75, 76, 77, 78, 79],
               [80, 81, 82, 83, 84, 85, 86, 87, 88, 89],
               [90, 91, 92, 93, 94, 95, 96, 97, 98, 99]])
```

```
In [100... mat>50
```

```
Out[100...] array([[False, False, False, False, False, False, False, False, False,
        False],
        [False, False, False, False, False, False, False, False, False,
        False],
        [False, False, False, False, False, False, False, False, False,
        False],
        [False, False, False, False, False, False, False, False, False,
        False],
        [False, False, False, False, False, False, False, False, False,
        False],
        [False, True, True, True, True, True, True, True, True,
        True],
        [ True, True, True, True, True, True, True, True, True,
        True],
        [ True, True, True, True, True, True, True, True, True,
        True],
        [ True, True, True, True, True, True, True, True, True,
        True],
        [ True, True, True, True, True, True, True, True, True,
        True]])
```

```
In [101...] mat[mat>50]
```

```
Out[101...] array([51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67,
        68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84,
        85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99])
```

```
In [102...] mat[mat>=50]
```

```
Out[102...] array([50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66,
        67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83,
        84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99])
```

```
In [103...] mat[mat!=50]
```

```
Out[103...] array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16,
        17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33,
        34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 51,
        52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68,
        69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85,
        86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99])
```

```
In [104...] mat[mat==50]
```

```
Out[104...] array([50])
```

```
In [105...] mat[mat<=50]
```

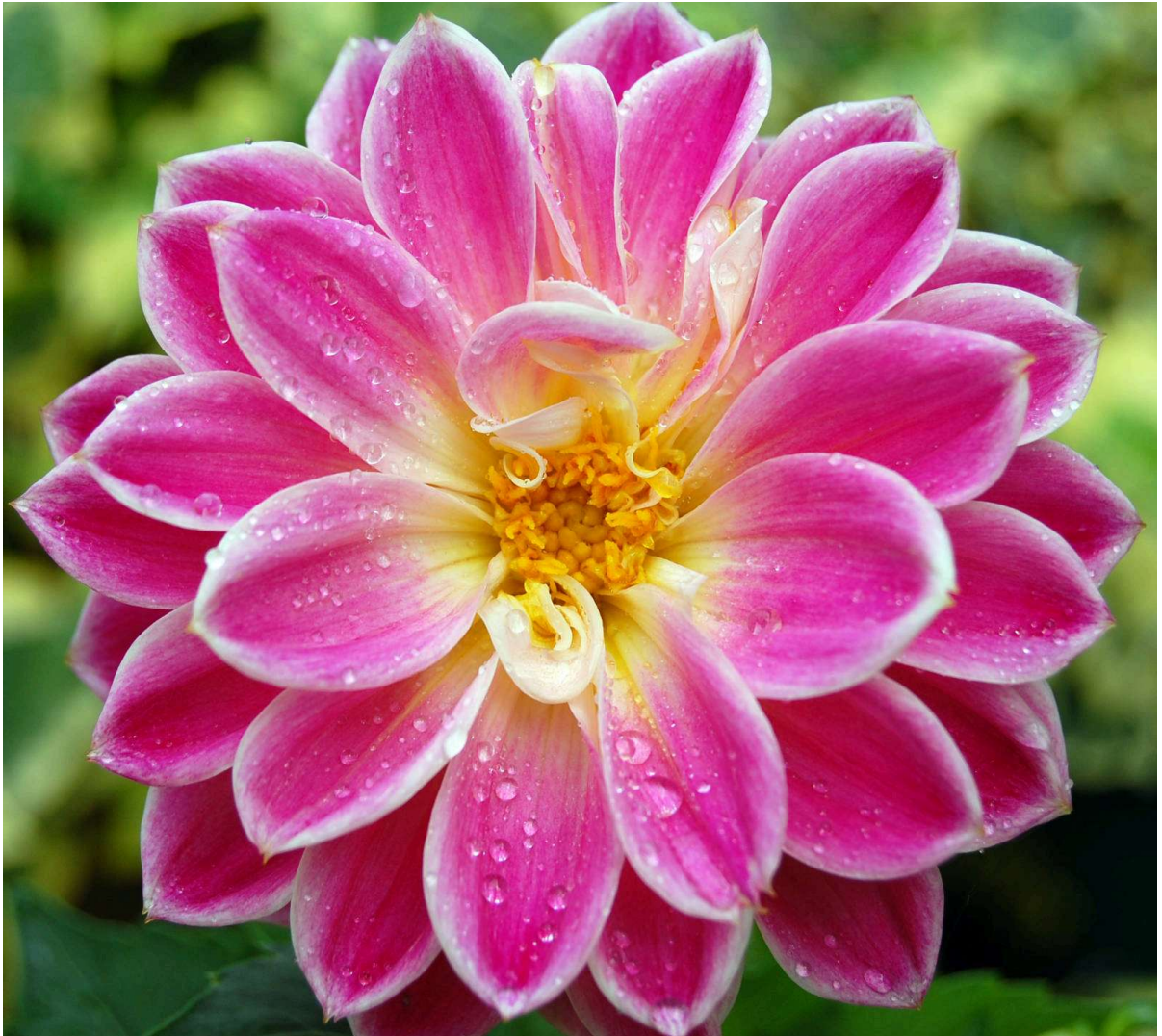
```
Out[105...] array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16,
        17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33,
        34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50])
```

Conversion of image to an array

```
In [106... from PIL import Image    ### PIL - PYTHON IMAGEING LIBRARY
```

```
In [107... image = Image.open(r"C:\Users\Shravani\Downloads\flower.jpg")  
image
```

Out[107...



```
In [108... type(image)
```

Out[108... PIL.JpegImagePlugin.JpegImageFile

```
In [109... flo_arr = np.asarray(image).astype(int)  
flo_arr
```

```

Out[109... array([[134, 174, 114],
                  [134, 174, 114],
                  [134, 174, 112],
                  ...,
                  [144, 169, 86],
                  [145, 170, 87],
                  [145, 170, 87]]],

          [[134, 174, 114],
           [134, 174, 114],
           [134, 174, 112],
           ...,
           [144, 169, 86],
           [144, 169, 86],
           [145, 170, 87]]],

          [[133, 173, 113],
           [134, 174, 114],
           [135, 175, 113],
           ...,
           [143, 168, 85],
           [144, 169, 86],
           [144, 169, 86]]],

          ...,

          [[ 32, 71, 24],
           [ 31, 69, 22],
           [ 28, 65, 21],
           ...,
           [ 97, 131, 57],
           [ 96, 130, 54],
           [ 96, 130, 53]]],

          [[ 30, 69, 22],
           [ 30, 68, 21],
           [ 26, 63, 19],
           ...,
           [ 97, 131, 57],
           [ 96, 130, 54],
           [ 96, 130, 53]]],

          [[ 29, 68, 21],
           [ 28, 66, 19],
           [ 25, 62, 18],
           ...,
           [ 97, 131, 57],
           [ 96, 130, 54],
           [ 96, 130, 53]]], shape=(1436, 1600, 3))

```

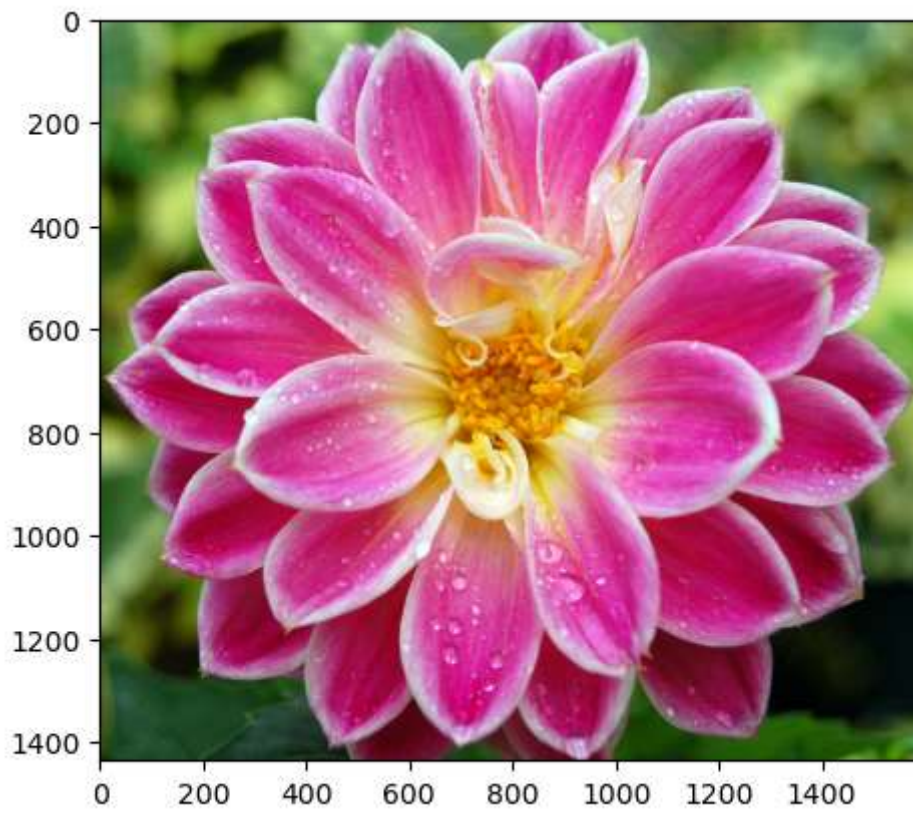
```
In [110... flo_arr.shape
```

```
Out[110... (1436, 1600, 3)
```

```
In [111... import matplotlib.pyplot as plt
```

```
In [112... plt.imshow(flo_arr)
```

```
Out[112... <matplotlib.image.AxesImage at 0x201ff719550>
```



```
In [115... flo_arr.shape
```

```
Out[115... (1436, 1600, 3)
```

```
In [116... image = Image.open(r"C:\Users\Shravani\Downloads\WhatsApp Image 2026-01-28 at 11.10  
image
```

Out[116...

In [117... `type(image)`Out[117... `PIL.JpegImagePlugin.JpegImageFile`In [118... `name_arr = np.asarray(image).astype(int)`
`name_arr`


```

Out[118... array([[0, 0, 0],
                  [0, 0, 0],
                  [0, 0, 0],
                  ...,
                  [0, 0, 0],
                  [0, 0, 0],
                  [0, 0, 0]],

                [[0, 0, 0],
                  [0, 0, 0],
                  [0, 0, 0],
                  ...,
                  [0, 0, 0],
                  [0, 0, 0],
                  [0, 0, 0]],

                [[0, 0, 0],
                  [0, 0, 0],
                  [0, 0, 0],
                  ...,
                  [0, 0, 0],
                  [0, 0, 0],
                  [0, 0, 0]],

                ...,

                [[0, 0, 0],
                  [0, 0, 0],
                  [0, 0, 0],
                  ...,
                  [0, 0, 0],
                  [0, 0, 0],
                  [0, 0, 0]],

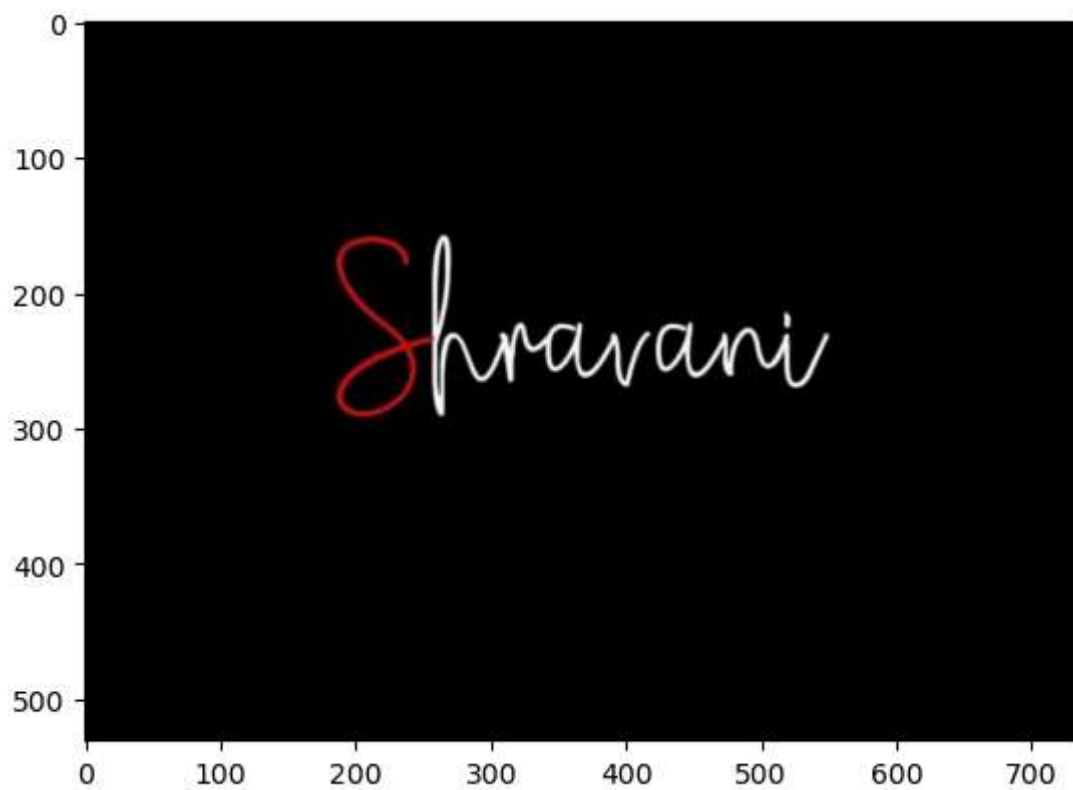
                [[0, 0, 0],
                  [0, 0, 0],
                  [0, 0, 0],
                  ...,
                  [0, 0, 0],
                  [0, 0, 0],
                  [0, 0, 0]],

                [[0, 0, 0],
                  [0, 0, 0],
                  [0, 0, 0],
                  ...,
                  [0, 0, 0],
                  [0, 0, 0],
                  [0, 0, 0]]], shape=(531, 736, 3))

```

```
In [119... plt.imshow(name_arr)
```

```
Out[119... <matplotlib.image.AxesImage at 0x2018364a0d0>
```



In []:

Numpy

- NumPy (Numerical Python) is a powerful Python library used for numerical computing, providing support for multi-dimensional arrays, matrices, and high-performance mathematical, statistical, and linear algebra operations.

Why we need NumPy (instead of normal Python lists)

- Faster than Python lists
- Uses less memory
- Supports vectorized operations (operate on whole arrays at once)

Real Time Application :

- Cleaning numeric data
- Handling missing values
- Performing fast calculations on datasets

Use the following import convention

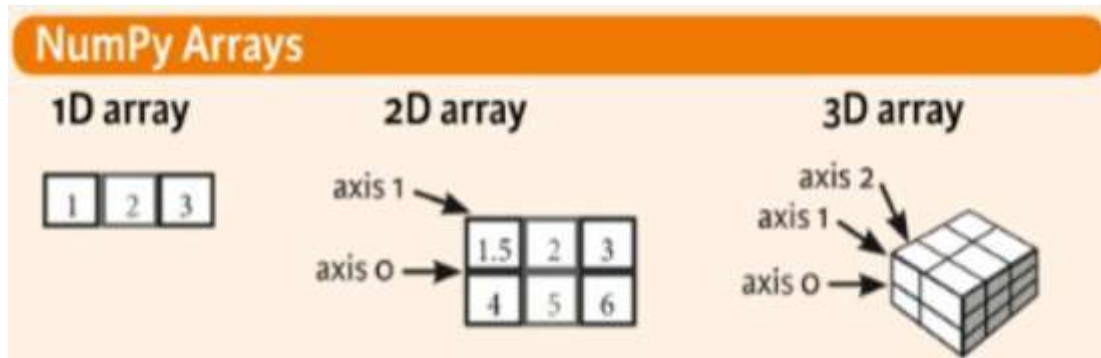
```
In [9]: import numpy as np
```

Numpy Arrays

```
In [11]: from PIL import Image
```

```
In [12]: image = Image.open(r"C:\Users\Shravani\Downloads\WhatsApp Image 2026-01-29 at 9.08.  
image
```

Out[12]:



Creating Arrays

1D array

```
In [25]: a = np.array([1, 2, 3])
print(a)
```

```
[1 2 3]
```

2D array with float dtype

```
In [24]: b = np.array([(1.5, 2, 3), (4, 5, 6)], dtype=float)
print(b)
```

```
[[1.5 2.  3. ]
 [4.  5.  6. ]]
```

ND array (mixed nested structure)

```
In [72]: c = np.array([[(1.5, 2, 3), (4, 5, 6)], [(3, 2, 1), (4, 5, 6)]], dtype=float)
print(c)
```

```
[[[1.5 2.  3. ]
  [4.  5.  6. ]]

 [[3.  2.  1. ]
  [4.  5.  6. ]]]
```

NumPy Array Operations & Functions

1) Array Creation Functions

Create an array of zeros using np.zeros()

- Creates an array filled with zeros

```
In [28]: np.zeros((3, 4))
```

```
Out[28]: array([[0., 0., 0., 0.],
               [0., 0., 0., 0.],
               [0., 0., 0., 0.]])
```

Create an array of ones using np.ones()

- Creates an array filled with ones

```
In [33]: np.ones((2, 3, 4))
```

```
Out[33]: array([[[1., 1., 1., 1.],
                 [1., 1., 1., 1.],
                 [1., 1., 1., 1.]],
               [[1., 1., 1., 1.],
                 [1., 1., 1., 1.],
                 [1., 1., 1., 1.]])
```

Create an array of evenly spaced values (step value) - np.arange()

- Creates numbers with a fixed step

```
In [43]: np.arange(10, 25, 5)
```

```
Out[43]: array([10, 15, 20])
```

Create an array of evenly spaced values using np.linspace()

- Creates evenly spaced values between two numbers

```
In [37]: np.linspace(0, 2, 9)
```

```
Out[37]: array([0. , 0.25, 0.5 , 0.75, 1.  , 1.25, 1.5 , 1.75, 2.  ])
```

Create a 2x2 identity matrix using np.eye()

- Creates an identity matrix

```
In [38]: np.eye(2)
```

```
Out[38]: array([[1., 0.],
               [0., 1.]])
```

Create an array with random values using np.random.random()

- Creates random values between 0 and 1

```
In [115... np.random.random((2, 2))
```

```
Out[115... array([[0.4236548 , 0.64589411],
        [0.43758721, 0.891773  ]])
```

Create an empty array using np.empty()

- Creates an array without initializing values

```
In [40]: np.empty((3, 2))
```

```
Out[40]: array([[1.5, 2. ],
               [3. , 4. ],
               [5. , 6. ]])
```

2) Array Manipulation Functions

reshape() an array

- reshape() changes the shape of an array without changing its data.

```
In [68]: a1 = np.array([1,2,3])
        reshaped = np.reshape(a1,(3,1)) ### # Reshape to 3*1
        print("Reshaped array:", reshaped)
```

```
Reshaped array: [[1]
                 [2]
                 [3]]
```

flatten an array using np.ravel()

- np.ravel() flattens a multi-dimensional array into a 1D array.

"Flattening" means we take all the elements row by row and put them into a single line.

```
In [69]: f1 = np.array([[1, 2], [3, 4]])
        flattened = np.ravel(f1) # Flatten to 1D array
        print("Flattened array:", flattened)
```

```
Flattened array: [1 2 3 4]
```

Transpose the array

- Transpose means swap rows and columns.

Row 1 becomes Column 1, Row 2 becomes Column 2.

```
In [70]: e1 = np.array([[1, 2], [3, 4]])
transposed = np.transpose(e1)
print("Transposed array:\n", transposed)
```

Transposed array:
[[1 3]
 [2 4]]

Vertical Stacking of Arrays using np.vstack

- np.vstack() stacks arrays row-wise, creating a 2D array from 1D arrays.

```
In [71]: a2 = np.array([1, 2])
b2 = np.array([3, 4])
stacked = np.vstack([a2, b2]) # Stack a and b vertically
print("Stacked arrays:\n", stacked)
```

Stacked arrays:
[[1 2]
 [3 4]]

Horizontal Stacking of Arrays using np.hstack()

- np.hstack() stacks arrays side by side (column-wise for 1D arrays).

```
In [73]: a2 = np.array([1, 2])
b2 = np.array([3, 4])
stacked = np.hstack([a2, b2]) # Stack a and b vertically
print("Stacked arrays:\n", stacked)
```

Stacked arrays:
[1 2 3 4]

3) Array Mathematical Functions

Adding two arrays using np.add()

```
In [74]: g = np.array([1, 2, 3, 4])
added = np.add(g, 2) # Add 2 to each element
print("Added 2 to g:", added)
```

Added 2 to g: [3 4 5 6]

Square each element using np.power()

```
In [77]: g = np.array([1, 2, 3, 4])
squared = np.power(g, 2) # Square each element
print("Squared g:", squared)
```

Squared g: [1 4 9 16]

Square root of each element using np.sqrt()

```
In [79]: g = np.array([1, 2, 3, 4])
sqrt_val = np.sqrt(g) # Square root of each element
print("Square root of g:", sqrt_val)
```

Square root of g: [1. 1.41421356 1.73205081 2.]

Dot Product of Arrays using np.dot()

- np.dot(array1, array2) calculates the sum of products of corresponding elements.

```
In [81]: a1 = np.array([1, 2, 3])
a3 = np.array([4, 5, 6]) # Dot product=(1*4)+(2*5)+(3*6)=4+10+18=32

dot_product = np.dot(a1, a3)
print("Dot product of a1 and a3:", dot_product)
```

Dot product of a1 and a3: 32

4) Array Statistical Functions

np.mean()

- Average of an Array in NumPy

```
In [82]: s = np.array([1, 5, 3, 2])
mean = np.mean(s)
print("Mean of s:", mean)
```

Mean of s: 2.75

np.var()

- Variance measures how much the values in a dataset spread out from the mean

```
In [85]: s = np.array([1, 5, 3, 2])
mean = np.var(s) ## Var - variance
print("Mean of s:", mean)
```

Mean of s: 2.1875

Standard deviation of an array using np.std()

- np.std() calculates how much the data values deviate from the mean in a NumPy array.

```
In [87]: s = np.array([1, 5, 3, 2])
std_dev = np.std(s)
```

```
print("Standard deviation of s:", std_dev)
```

Standard deviation of s: 1.479019945774904

Minimum element of an array

```
In [90]: s = np.array([1, 5, 3, 2])
         minimum = np.min(s)
         print("Min of s:", minimum)
```

Min of s: 1

Maximum element of an array

```
In [92]: s = np.array([1, 5, 3, 2])
         maximum = np.max(s)
         print("Max of s:", maximum)
```

Max of s: 5

5) Linear Algebra Functions

Create a matrix

```
In [100... matrix = np.array([[1, 2], [3, 4]])
          matrix
```

```
Out[100... array([[1, 2],
                  [3, 4]])
```

6) Array Random Sampling Functions

```
In [101... # Generate random values between 0 and 1

random_vals = np.random.rand(3) # Array of 3 random values between 0 and 1
print("Random values:", random_vals)
```

Random values: [0.40284162 0.48059606 0.4928633]

```
In [102... # Set seed for reproducibility

np.random.seed(0)

# Generate random values between 0 and 1
random_vals = np.random.rand(3) # Array of 3 random values between 0 and 1
print("Random values:", random_vals)
```

Random values: [0.5488135 0.71518937 0.60276338]

```
In [103... # Generate random integers
```

```
rand_ints = np.random.randint(0, 10, size=5) # Random integers between 0 and 10
print("Random integers:", rand_ints)
```

Random integers: [3 7 9 3 5]

```
In [104... # Set seed for reproducibility

np.random.seed(0)

# Generate random integers
rand_ints = np.random.randint(0, 10, size=5) # Random integers between 0 and 10
print("Random integers:", rand_ints)
```

Random integers: [5 0 3 3 7]

7) Array Boolean & Logical Operators

Check if all elements are True

all

```
In [105... logical_test = np.array([True, False, True])
all_true = np.all(logical_test) # Check if all are True
print("All elements True:", all_true)
```

All elements True: False

Check if any elements are True

any

```
In [106... any_true = np.any(logical_test) # Check if any are True
print("Any elements True:", any_true)
```

Any elements True: True

8) Set Operations

Intersection of two arrays

```
In [107... set_a = np.array([3, 4, 5, 6])
set_b = np.array([5, 6, 7, 8])
intersection = np.intersect1d(set_a, set_b)
print("Intersection of a and b:", intersection)
```

Intersection of a and b: [5 6]

Union of two arrays


```
In [109... set_a = np.array([3, 4, 5, 6])
set_b = np.array([5, 6, 7, 8])
union = np.union1d(set_a, set_b)
print("Union of a and b:", union)
```

Union of a and b: [3 4 5 6 7 8]

9) Array Attribute Functions

```
In [113... # Array attributes
a = np.array([3, 4, 5])
shape = a.shape # Shape of the array
size = a.size # Number of elements
dimensions = a.ndim # Number of dimensions
dtype = a.dtype # Data type of the array

print("Shape of a:", shape)
print("Size of a:", size)
print("Number of dimensions of a:", dimensions)
print("Data type of a:", dtype)
```

Shape of a: (3,)

Size of a: 3

Number of dimensions of a: 1

Data type of a: int64

In []: